

api_unified.py

Archivo:

/Users/adan/Documents/documentos_blatam/truthgpt_collected/integration_code/production_code/api_unified.py

Líneas de código: 806

Fecha de generación: 2025-11-24 10:02:49

```
1 | #!/usr/bin/env python3
2 | """
3 | API REST Unificada
4 | =====
5 |
6 | API REST completa para todos los módulos del sistema.
7 |
8 | Este módulo proporciona:
9 | - API REST unificada con FastAPI
10 | - Endpoints para memory, redundancy, pipeline, chat
11 | - Factory function para crear la aplicación
12 | - Funciones de utilidad para verificar disponibilidad
13 | - Validación robusta de requests
14 |
15 | Ejemplo:
16 |     >>> app = create_api_app()
17 |     >>> # 0 usar directamente: uvicorn.run(app, host="0.0.0.0", port=8000)
18 | """
19 |
20 | __version__ = '2.0.0'
21 |
22 | import asyncio
23 | from contextlib import asynccontextmanager
24 | from datetime import datetime
25 | from typing import Any, Callable, Dict, List, Optional
26 |
27 | from fastapi import Depends, FastAPI, HTTPException, Request
28 | from fastapi.middleware.cors import CORSMiddleware
29 | from fastapi.responses import FileResponse, JSONResponse
30 | from pydantic import BaseModel, Field
31 | import uvicorn
32 |
33 | from api_utils import (
34 |     validate_episode_data,
35 |     validate_items_data,
36 |     validate_k_value,
37 |     validate_priority,
38 |     validate_query_data,
39 |     validate_similarity_threshold,
40 | )
41 | from core.utils import setup_logger
42 |
43 | logger = setup_logger(__name__)
44 |
45 | try:
46 |     from integration_pipeline import create_integrated_pipeline
47 |     PIPELINE_AVAILABLE = True
48 | except ImportError:
49 |     create_integrated_pipeline = None
50 |     PIPELINE_AVAILABLE = False
```

```

51 |
52 | try:
53 |     from config_manager import get_config_manager
54 |     CONFIG_MANAGER_AVAILABLE = True
55 | except ImportError:
56 |     get_config_manager = None
57 |     CONFIG_MANAGER_AVAILABLE = False
58 |
59 | try:
60 |     from monitoring_system import get_system_monitor
61 |     MONITORING_AVAILABLE = True
62 | except ImportError:
63 |     get_system_monitor = None
64 |     MONITORING_AVAILABLE = False
65 |
66 |
67 | @asynccontextmanager
68 | async def lifespan(app: FastAPI):
69 |     """Gestiona recursos globales y los expone vía app.state."""
70 |     app.state.pipeline = None
71 |     app.state.config_manager = None
72 |     app.state.monitor = None
73 |     app.state._resource_locks = {}
74 |     app.state._resource_errors = {}
75 |
76 |     try:
77 |         if PIPELINE_AVAILABLE and create_integrated_pipeline is not None:
78 |             pipeline = await _ensure_state_resource(
79 |                 app,
80 |                 "pipeline",
81 |                 lambda: create_integrated_pipeline(
82 |                     enable_memory=True,
83 |                     enable_redundancy=True
84 |                 ),
85 |                 True,
86 |                 "pipeline"
87 |             )
88 |             if pipeline:
89 |                 logger.info("Pipeline inicializado en lifespan")
90 |         except Exception as exc:
91 |             logger.error(f"No se pudo inicializar pipeline: {exc}", exc_info=True)
92 |
93 |     try:
94 |         if CONFIG_MANAGER_AVAILABLE and get_config_manager is not None:
95 |             config_manager = await _ensure_state_resource(
96 |                 app,
97 |                 "config_manager",
98 |                 get_config_manager,
99 |                 True,
100 |                 "config manager"

```

```

101 |         )
102 |         if config_manager:
103 |             logger.info("Config manager inicializado en lifespan")
104 |     except Exception as exc:
105 |         logger.error(f"No se pudo inicializar config manager: {exc}", exc_info=True)
106 |
107 |     try:
108 |         if MONITORING_AVAILABLE and get_system_monitor is not None:
109 |             monitor = await _ensure_state_resource(
110 |                 app,
111 |                 "monitor",
112 |                 get_system_monitor,
113 |                 True,
114 |                 "monitor"
115 |             )
116 |             if monitor:
117 |                 logger.info("Monitor inicializado en lifespan")
118 |     except Exception as exc:
119 |         logger.error(f"No se pudo inicializar monitor: {exc}", exc_info=True)
120 |
121 |     try:
122 |         yield
123 |     finally:
124 |         pipeline = getattr(app.state, "pipeline", None)
125 |         if pipeline and hasattr(pipeline, "shutdown"):
126 |             try:
127 |                 pipeline.shutdown()
128 |             except Exception as exc: # pragma: no cover
129 |                 logger.warning(f"Error apagando pipeline: {exc}")
130 |
131 |         monitor = getattr(app.state, "monitor", None)
132 |         if monitor and hasattr(monitor, "shutdown"):
133 |             try:
134 |                 monitor.shutdown()
135 |             except Exception as exc: # pragma: no cover
136 |                 logger.warning(f"Error apagando monitor: {exc}")
137 |
138 |
139 | # Crear aplicación FastAPI (instancia por defecto)
140 | app = FastAPI(
141 |     title="Production Code API",
142 |     description="API REST unificada para todos los módulos",
143 |     version=__version__,
144 |     lifespan=lifespan
145 | )
146 |
147 | # CORS
148 | app.add_middleware(
149 |     CORSMiddleware,
150 |     allow_origins=["*"],

```

```

151 |     allow_credentials=True,
152 |     allow_methods=["*"],
153 |     allow_headers=["*"],
154 | )
155 |
156 | # Middleware adicional
157 | try:
158 |     from api_middleware import LoggingMiddleware, MetricsMiddleware, ErrorHandlingMiddleware
159 |     app.add_middleware(LoggingMiddleware)
160 |     app.add_middleware(MetricsMiddleware)
161 |     app.add_middleware(ErrorHandlingMiddleware)
162 |     logger.info("Middleware adicional cargado")
163 | except ImportError:
164 |     logger.warning("Middleware adicional no disponible")
165 |
166 | # Modelos Pydantic
167 | class MemoryStoreRequest(BaseModel):
168 |     episode: List[float] = Field(..., description="Episodio como lista")
169 |     metadata: Optional[Dict[str, Any]] = Field(None, description="Metadata adicional")
170 |     tags: Optional[List[str]] = Field(None, description="Tags")
171 |     priority: Optional[float] = Field(1.0, description="Prioridad")
172 |
173 |
174 | class MemoryRetrieveRequest(BaseModel):
175 |     query: List[float] = Field(..., description="Query como lista")
176 |     k: Optional[int] = Field(10, description="Número de episodios a recuperar")
177 |
178 |
179 | class RedundancyProcessRequest(BaseModel):
180 |     items: List[List[List[float]]] = Field(..., description="Items como lista 3D")
181 |     similarity_threshold: Optional[float] = Field(0.85, description="Umbral de similitud")
182 |
183 |
184 | class PipelineProcessRequest(BaseModel):
185 |     data: List[List[List[float]]] = Field(..., description="Datos como lista 3D")
186 |     use_memory: Optional[bool] = Field(True, description="Usar memoria")
187 |     use_redundancy: Optional[bool] = Field(True, description="Usar redundancia")
188 |
189 |
190 | class ChatRequest(BaseModel):
191 |     message: str = Field(..., description="Mensaje del usuario")
192 |     conversation_id: Optional[str] = Field(None, description="ID de conversación")
193 |
194 |
195 | class DocumentConvertRequest(BaseModel):
196 |     data: Union[Dict[str, Any], List[Any], str] = Field(..., description="Datos a convertir")
197 |     format: str = Field(..., description="Formato de salida: pdf, docx, xlsx")
198 |     title: Optional[str] = Field("Documento", description="Título del documento")
199 |     method: Optional[str] = Field("auto", description="Método específico a usar (auto para selección automática)")
200 |     output_filename: Optional[str] = Field(None, description="Nombre del archivo de salida (sin extensión)")

```

```

201 |
202 |
203 | class DocumentConvertMultipleRequest(BaseModel):
204 |     data: Union[Dict[str, Any], List[Any], str] = Field(..., description="Datos a convertir")
205 |     formats: List[str] = Field(..., description="Lista de formatos: ['pdf', 'docx', 'xlsx']")
206 |     title: Optional[str] = Field("Documento", description="Título del documento")
207 |     output_filename: Optional[str] = Field(None, description="Nombre base del archivo (sin extensión)")
208 |
209 |
210 | # Modelos de Respuesta
211 | class MemoryStoreResponse(BaseModel):
212 |     success: bool = Field(..., description="Indica si el almacenamiento fue exitoso")
213 |     episodes: int = Field(..., description="Número total de episodios almacenados")
214 |
215 |
216 | class MemoryRetrieveResponse(BaseModel):
217 |     retrieved: List[List[float]] = Field(..., description="Episodios recuperados")
218 |     weights: List[float] = Field(..., description="Pesos de similitud")
219 |     count: int = Field(..., description="Número de episodios recuperados")
220 |
221 |
222 | class RedundancyProcessResponse(BaseModel):
223 |     unique_items: List[List[List[float]]] = Field(..., description="Items únicos después de procesar")
224 |     stats: Dict[str, Any] = Field(..., description="Estadísticas del procesamiento")
225 |     reduction_rate: float = Field(..., description="Tasa de reducción de redundancia")
226 |
227 |
228 | class PipelineProcessResponse(BaseModel):
229 |     output: List[List[List[float]]] = Field(..., description="Datos procesados")
230 |     metadata: Dict[str, Any] = Field(..., description="Metadata del procesamiento")
231 |     output_shape: List[int] = Field(..., description="Forma del tensor de salida")
232 |
233 |
234 | class StatusResponse(BaseModel):
235 |     timestamp: str = Field(..., description="Timestamp ISO del estado")
236 |     resources: Dict[str, Dict[str, Any]] = Field(..., description="Estado de recursos")
237 |     modules: Dict[str, bool] = Field(..., description="Disponibilidad de módulos")
238 |
239 |
240 | # Dependencias
241 | async def get_pipeline_dependency(request: Request):
242 |     """Obtiene o inicializa el pipeline de manera perezosa."""
243 |     if not PIPELINE_AVAILABLE or create_integrated_pipeline is None:
244 |         _raise_unavailable("Pipeline")
245 |
246 |     pipeline = await _ensure_state_resource(
247 |         request.app,
248 |         "pipeline",
249 |         lambda: create_integrated_pipeline(enable_memory=True, enable_redundancy=True),
250 |         True,

```

```

251 |         "pipeline"
252 |     )
253 |     if pipeline is None:
254 |         _raise_unavailable("Pipeline", _get_resource_error(request.app, "pipeline"))
255 |     return pipeline
256 |
257 |
258 | async def get_config_manager_dependency(request: Request):
259 |     """Obtiene o inicializa el gestor de configuración."""
260 |     if not CONFIG_MANAGER_AVAILABLE or get_config_manager is None:
261 |         _raise_unavailable("Config manager")
262 |
263 |     config_manager = await _ensure_state_resource(
264 |         request.app,
265 |         "config_manager",
266 |         get_config_manager,
267 |         True,
268 |         "config manager"
269 |     )
270 |     if config_manager is None:
271 |         _raise_unavailable("Config manager", _get_resource_error(request.app, "config_manager"))
272 |     return config_manager
273 |
274 |
275 | async def get_monitor_dependency(request: Request):
276 |     """Obtiene o inicializa el monitor del sistema."""
277 |     if not MONITORING_AVAILABLE or get_system_monitor is None:
278 |         _raise_unavailable("Monitor")
279 |
280 |     monitor = await _ensure_state_resource(
281 |         request.app,
282 |         "monitor",
283 |         get_system_monitor,
284 |         True,
285 |         "monitor"
286 |     )
287 |     if monitor is None:
288 |         _raise_unavailable("Monitor", _get_resource_error(request.app, "monitor"))
289 |     return monitor
290 |
291 |
292 | # Helpers para reducir duplicación
293 | def handle_api_error(operation_name: str, error: Exception) -> HTTPException:
294 |     """Helper para manejar errores de API de manera consistente."""
295 |     logger.error(f"Error en {operation_name}: {error}")
296 |     return HTTPException(status_code=500, detail=str(error))
297 |
298 |
299 | def check_module_available(module_enabled: bool, module_instance: Any, module_name: str) -> None:
300 |     """Verifica que un módulo esté disponible."""

```

```

301 |         if not module_enabled or not module_instance:
302 |             raise HTTPException(status_code=503, detail=f"{module_name} module not available")
303 |
304 |
305 | async def _ensure_state_resource(
306 |     app: FastAPI,
307 |     attr_name: str,
308 |     factory: Callable[[], Any],
309 |     enabled: bool,
310 |     resource_label: str,
311 | ) -> Optional[Any]:
312 |     """Inicializa recursos compartidos de manera perezosa y segura."""
313 |     if not enabled:
314 |         return None
315 |
316 |     resource = getattr(app.state, attr_name, None)
317 |     if resource is not None:
318 |         return resource
319 |
320 |     locks = getattr(app.state, "_resource_locks", None)
321 |     if locks is None:
322 |         locks = {}
323 |         app.state._resource_locks = locks
324 |
325 |     lock = locks.get(attr_name)
326 |     if lock is None:
327 |         lock = asyncio.Lock()
328 |         locks[attr_name] = lock
329 |
330 |     async with lock:
331 |         resource = getattr(app.state, attr_name, None)
332 |         if resource is not None:
333 |             return resource
334 |
335 |         try:
336 |             resource = await asyncio.to_thread(factory)
337 |         except Exception as exc:
338 |             logger.error(f"No se pudo inicializar {resource_label}: {exc}", exc_info=True)
339 |             errors = getattr(app.state, "_resource_errors", None)
340 |             if errors is None:
341 |                 errors = {}
342 |                 app.state._resource_errors = errors
343 |             errors[attr_name] = exc
344 |             return None
345 |
346 |         setattr(app.state, attr_name, resource)
347 |         errors = getattr(app.state, "_resource_errors", None)
348 |         if errors and attr_name in errors:
349 |             errors.pop(attr_name, None)
350 |         logger.info("%s inicializado correctamente", resource_label)

```

```

351 |         return resource
352 |
353 |
354 | def _get_resource_error(app: FastAPI, attr_name: str) -> Optional[str]:
355 |     """Obtiene el último error registrado para un recurso."""
356 |     errors = getattr(app.state, "_resource_errors", None)
357 |     if not errors:
358 |         return None
359 |     error = errors.get(attr_name)
360 |     return str(error) if error else None
361 |
362 |
363 | def _raise_unavailable(resource_label: str, error: Optional[str] = None) -> None:
364 |     """Lanza HTTP 503 con información opcional de error."""
365 |     detail = f"{resource_label} not available"
366 |     if error:
367 |         detail = f"{detail}: {error}"
368 |     raise HTTPException(status_code=503, detail=detail)
369 |
370 |
371 | def _resource_status(app: FastAPI, attr_name: str, enabled: bool) -> Dict[str, Any]:
372 |     """Constuye snapshot de estado para endpoints de monitoreo."""
373 |     instance = getattr(app.state, attr_name, None)
374 |     error = _get_resource_error(app, attr_name)
375 |     return {
376 |         "enabled": enabled,
377 |         "initialized": bool(instance),
378 |         "error": error
379 |     }
380 |
381 |
382 | # Endpoints de Memory
383 | @app.post("/api/v1/memory/store", tags=["memory"], response_model=MemoryStoreResponse)
384 | async def memory_store(request: MemoryStoreRequest, pipeline=Depends(get_pipeline_dependency)):
385 |     """Almacena un episodio en memoria."""
386 |     try:
387 |         check_module_available(pipeline.enable_memory, pipeline.memory_system, "Memory")
388 |         episode = validate_episode_data(request.episode)
389 |         priority = validate_priority(request.priority)
390 |
391 |         success = pipeline.memory_system.store_episode(
392 |             episode,
393 |             metadata=request.metadata,
394 |             tags=request.tags if request.tags else None,
395 |             priority=priority
396 |         )
397 |
398 |         return MemoryStoreResponse(
399 |             success=success,
400 |             episodes=len(pipeline.memory_system.episodic_memory)

```



```

401 |         )
402 |     except HTTPException:
403 |         raise
404 |     except Exception as e:
405 |         raise handle_api_error("almacenando episodio", e)
406 |
407 |
408 | @app.post("/api/v1/memory/retrieve", tags=["memory"], response_model=MemoryRetrieveResponse)
409 | async def memory_retrieve(request: MemoryRetrieveRequest, pipeline=Depends(get_pipeline_dependency)):
410 |     """Recupera episodios de memoria."""
411 |     try:
412 |         check_module_available(pipeline.enable_memory, pipeline.memory_system, "Memory")
413 |         query = validate_query_data(request.query)
414 |         k_value = validate_k_value(request.k)
415 |
416 |         retrieved, weights = pipeline.memory_system.retrieve_episodes(query, k=k_value)
417 |
418 |         return MemoryRetrieveResponse(
419 |             retrieved=retrieved.tolist(),
420 |             weights=weights.tolist(),
421 |             count=retrieved.shape[0]
422 |         )
423 |     except HTTPException:
424 |         raise
425 |     except Exception as e:
426 |         raise handle_api_error("recuperando episodios", e)
427 |
428 |
429 | @app.get("/api/v1/memory/stats", tags=["memory"])
430 | async def memory_stats(pipeline=Depends(get_pipeline_dependency)):
431 |     """Obtiene estadísticas de memoria."""
432 |     try:
433 |         check_module_available(pipeline.enable_memory, pipeline.memory_system, "Memory")
434 |
435 |         stats = pipeline.memory_system.get_episodic_stats()
436 |         return stats
437 |     except HTTPException:
438 |         raise
439 |     except Exception as e:
440 |         raise handle_api_error("obteniendo stats de memoria", e)
441 |
442 |
443 | # Endpoints de Redundancy
444 | @app.post("/api/v1/redundancy/process", tags=["redundancy"], response_model=RedundancyProcessResponse)
445 | async def redundancy_process(request: RedundancyProcessRequest, pipeline=Depends(get_pipeline_dependency)):
446 |     """Procesa datos eliminando redundancias."""
447 |     try:
448 |         check_module_available(pipeline.enable_redundancy, pipeline.redundancy_suppressor, "Redundancy")
449 |         items = validate_items_data(request.items, name="items")
450 |         similarity_threshold = validate_similarity_threshold(request.similarity_threshold)

```

```

451 |
452 |         unique_items, stats = pipeline.redundancy_suppressor.process_bulk(
453 |             items,
454 |             similarity_threshold=similarity_threshold
455 |         )
456 |
457 |         return RedundancyProcessResponse(
458 |             unique_items=unique_items.tolist(),
459 |             stats=stats,
460 |             reduction_rate=stats.get('reduction_rate', 0.0)
461 |         )
462 |     except HTTPException:
463 |         raise
464 |     except Exception as e:
465 |         raise handle_api_error("procesando redundancia", e)
466 |
467 |
468 | @app.get("/api/v1/redundancy/stats", tags=["redundancy"])
469 | async def redundancy_stats(pipeline=Depends(get_pipeline_dependency)):
470 |     """Obtiene estadísticas de redundancia."""
471 |     try:
472 |         check_module_available(pipeline.enable_redundancy, pipeline.redundancy_suppressor, "Redundancy")
473 |
474 |         stats = pipeline.redundancy_suppressor.get_metrics()
475 |         return stats
476 |     except HTTPException:
477 |         raise
478 |     except Exception as e:
479 |         raise handle_api_error("obteniendo stats de redundancia", e)
480 |
481 |
482 | # Endpoints de Pipeline
483 | @app.post("/api/v1/pipeline/process", tags=["pipeline"], response_model=PipelineProcessResponse)
484 | async def pipeline_process(request: PipelineProcessRequest, pipeline=Depends(get_pipeline_dependency)):
485 |     """Procesa datos a través del pipeline."""
486 |     try:
487 |         data = validate_items_data(request.data, name="data")
488 |         output, metadata = pipeline.process_pipeline(
489 |             data,
490 |             use_memory=request.use_memory,
491 |             use_redundancy=request.use_redundancy
492 |         )
493 |
494 |         return PipelineProcessResponse(
495 |             output=output.tolist(),
496 |             metadata=metadata,
497 |             output_shape=list(output.shape)
498 |         )
499 |     except HTTPException:
500 |         raise

```

```

501 |     except Exception as e:
502 |         raise handle_api_error("procesando pipeline", e)
503 |
504 |
505 | @app.get("/api/v1/pipeline/stats", tags=["pipeline"])
506 | async def pipeline_stats(pipeline=Depends(get_pipeline_dependency)):
507 |     """Obtiene estadísticas del pipeline."""
508 |     try:
509 |         stats = pipeline.get_pipeline_stats()
510 |         return stats
511 |     except HTTPException:
512 |         raise
513 |     except Exception as e:
514 |         raise handle_api_error("obteniendo stats del pipeline", e)
515 |
516 |
517 | # Endpoints de Chat
518 | @app.post("/api/v1/chat", tags=["chat"])
519 | async def chat(request: ChatRequest, pipeline=Depends(get_pipeline_dependency)):
520 |     """Chat con memoria integrada."""
521 |     try:
522 |         check_module_available(pipeline.enable_chat, pipeline.chat_engine, "Chat")
523 |
524 |         if not request.message or not request.message.strip():
525 |             raise HTTPException(status_code=400, detail="Message cannot be empty")
526 |
527 |         if len(request.message) > 10000:
528 |             raise HTTPException(status_code=400, detail="Message too long (max 10000 characters)")
529 |
530 |         response = pipeline.chat_with_memory(
531 |             request.message,
532 |             conversation_id=request.conversation_id
533 |         )
534 |
535 |         return response
536 |     except HTTPException:
537 |         raise
538 |     except Exception as e:
539 |         raise handle_api_error("procesando chat", e)
540 |
541 |
542 | # Endpoint de estado general
543 | @app.get("/api/v1/status", tags=["monitoring"], response_model=StatusResponse)
544 | async def service_status(request: Request):
545 |     """Resumen de disponibilidad y errores de recursos."""
546 |     resources = {
547 |         "pipeline": _resource_status(request.app, "pipeline", PIPELINE_AVAILABLE),
548 |         "config_manager": _resource_status(request.app, "config_manager", CONFIG_MANAGER_AVAILABLE),
549 |         "monitor": _resource_status(request.app, "monitor", MONITORING_AVAILABLE),
550 |     }

```

```

551 |         return StatusResponse(
552 |             timestamp=datetime.now().isoformat(),
553 |             resources=resources,
554 |             modules=get_available_modules()
555 |         )
556 |
557 |
558 | # Endpoints de Config
559 | @app.get("/api/v1/config", tags=["config"])
560 | async def get_config(module: Optional[str] = None, config_manager=Depends(get_config_manager_dependency)):
561 |     """Obtiene configuración."""
562 |     try:
563 |         if config_manager is None:
564 |             raise HTTPException(status_code=503, detail="Config manager not initialized")
565 |
566 |         if module:
567 |             from config_manager import ModuleType
568 |             module_type = ModuleType(module) if module in [m.value for m in ModuleType] else None
569 |             if module_type:
570 |                 config = config_manager.get_config(module_type)
571 |             else:
572 |                 config = config_manager.get_config(module)
573 |         else:
574 |             config = config_manager.get_all_configs()
575 |
576 |         return config
577 |     except HTTPException:
578 |         raise
579 |     except Exception as e:
580 |         raise handle_api_error("obteniendo config", e)
581 |
582 |
583 | @app.put("/api/v1/config/{module}", tags=["config"])
584 | async def update_config(module: str, config: Dict[str, Any], config_manager=Depends(
585 |     get_config_manager_dependency)):
586 |     """Actualiza configuración."""
587 |     try:
588 |         if config_manager is None:
589 |             raise HTTPException(status_code=503, detail="Config manager not initialized")
590 |
591 |         from config_manager import ModuleType
592 |         module_type = ModuleType(module) if module in [m.value for m in ModuleType] else None
593 |         if module_type:
594 |             config_manager.set_config(module_type, config)
595 |         else:
596 |             config_manager.set_config(module, config)
597 |
598 |         return {"success": True, "message": f"Config updated for {module}"}
599 |     except HTTPException:
600 |         raise
601 |     except Exception as e:

```

```

601 |         raise handle_api_error("actualizando config", e)
602 |
603 |
604 | # Endpoints de Monitoring
605 | @app.get("/api/v1/monitor/status", tags=["monitoring"])
606 | async def monitor_status(monitor=Depends(get_monitor_dependency)):
607 |     """Obtiene estado del sistema."""
608 |     try:
609 |         if monitor is None:
610 |             raise HTTPException(status_code=503, detail="Monitor not initialized")
611 |
612 |         status = monitor.get_system_status()
613 |         return status
614 |     except HTTPException:
615 |         raise
616 |     except Exception as e:
617 |         raise handle_api_error("obteniendo status del monitor", e)
618 |
619 |
620 | @app.get("/api/v1/monitor/health", tags=["monitoring"])
621 | async def monitor_health(monitor=Depends(get_monitor_dependency)):
622 |     """Obtiene health checks."""
623 |     try:
624 |         if monitor is None:
625 |             raise HTTPException(status_code=503, detail="Monitor not initialized")
626 |
627 |         health = monitor.health_monitor.get_overall_health()
628 |         return health
629 |     except HTTPException:
630 |         raise
631 |     except Exception as e:
632 |         raise handle_api_error("obteniendo health", e)
633 |
634 |
635 | @app.get("/api/v1/monitor/metrics", tags=["monitoring"])
636 | async def monitor_metrics(monitor=Depends(get_monitor_dependency)):
637 |     """Obtiene métricas."""
638 |     try:
639 |         if monitor is None:
640 |             raise HTTPException(status_code=503, detail="Monitor not initialized")
641 |
642 |         metrics = monitor.metrics_collector.get_all_metrics()
643 |         return metrics
644 |     except HTTPException:
645 |         raise
646 |     except Exception as e:
647 |         raise handle_api_error("obteniendo metrics", e)
648 |
649 |
650 | # Endpoint raíz

```

```

651 | @app.get("/", tags=["info"])
652 | async def root():
653 |     """Endpoint raíz."""
654 |     return {
655 |         "name": "Production Code API",
656 |         "version": "2.0.0",
657 |         "status": "running",
658 |         "endpoints": {
659 |             "memory": "/api/v1/memory",
660 |             "redundancy": "/api/v1/redundancy",
661 |             "pipeline": "/api/v1/pipeline",
662 |             "chat": "/api/v1/chat",
663 |             "config": "/api/v1/config",
664 |             "monitor": "/api/v1/monitor"
665 |         }
666 |     }
667 |
668 |
669 | @app.get("/health", tags=["info"])
670 | async def health():
671 |     """Health check endpoint."""
672 |     return {"status": "healthy", "timestamp": datetime.now().isoformat()}
673 |
674 |
675 | @app.get("/dashboard", tags=["info"])
676 | async def dashboard():
677 |     """Sirve el dashboard HTML."""
678 |     from pathlib import Path
679 |     dashboard_path = Path(__file__).parent / "dashboard.html"
680 |     if dashboard_path.exists():
681 |         return FileResponse(dashboard_path)
682 |     else:
683 |         return JSONResponse({"error": "Dashboard not found"}, status_code=404)
684 |
685 |
686 | def get_available_modules() -> Dict[str, bool]:
687 |     """
688 |     Obtiene el estado de disponibilidad de todos los módulos de la API.
689 |
690 |     Returns:
691 |         Diccionario con el estado de disponibilidad de cada módulo.
692 |
693 |     Example:
694 |         >>> modules = get_available_modules()
695 |         >>> if modules['pipeline']:
696 |             ...     print("Pipeline disponible")
697 |     """
698 |     return {
699 |         'pipeline': PIPELINE_AVAILABLE,
700 |         'config_manager': CONFIG_MANAGER_AVAILABLE,

```

```

701 |         'monitoring': MONITORING_AVAILABLE
702 |     }
703 |
704 |
705 | def create_api_app(
706 |     enable_pipeline: bool = True,
707 |     enable_monitoring: bool = True,
708 |     cors_origins: Optional[List[str]] = None
709 | ) -> FastAPI:
710 |     """
711 |     Factory function para crear aplicación API con configuración personalizada.
712 |
713 |     Nota: Esta función retorna la instancia `app` existente. Los parámetros
714 |     se usan para validación y logging, pero todos los endpoints ya están
715 |     registrados en la instancia global `app`.
716 |
717 |     Args:
718 |         enable_pipeline: Habilitar endpoints del pipeline (default: True)
719 |         enable_monitoring: Habilitar endpoints de monitoreo (default: True)
720 |         cors_origins: Lista de orígenes permitidos para CORS (default: ["*"])
721 |
722 |     Returns:
723 |         Instancia de FastAPI configurada (la instancia global `app`).
724 |
725 |     Raises:
726 |         ValueError: Si los parámetros no son válidos.
727 |
728 |     Example:
729 |         >>> # API básica (retorna la instancia global)
730 |         >>> api_app = create_api_app()
731 |         >>> uvicorn.run(api_app, host="0.0.0.0", port=8000)
732 |         >>>
733 |         >>> # Verificar disponibilidad antes de usar
734 |         >>> modules = get_available_modules()
735 |         >>> if modules['pipeline']:
736 |         ...     api_app = create_api_app()
737 |     """
738 |     # Validación de entrada
739 |     if not isinstance(enable_pipeline, bool):
740 |         raise ValueError(f"enable_pipeline debe ser bool, recibido: {type(enable_pipeline).__name__}")
741 |     if not isinstance(enable_monitoring, bool):
742 |         raise ValueError(f"enable_monitoring debe ser bool, recibido: {type(enable_monitoring).__name__}")
743 |     if cors_origins is not None and not isinstance(cors_origins, list):
744 |         raise ValueError(f"cors_origins debe ser list o None, recibido: {type(cors_origins).__name__}")
745 |
746 |     try:
747 |         # Actualizar CORS si se proporciona configuración personalizada
748 |         if cors_origins is not None:
749 |             # Remover middleware CORS existente y añadir nuevo
750 |             app.user_middleware = [m for m in app.user_middleware if m.cls != CORSMiddleware]

```

```

751 |         app.add_middleware(
752 |             CORSMiddleware,
753 |             allow_origins=cors_origins,
754 |             allow_credentials=True,
755 |             allow_methods=["*"],
756 |             allow_headers=["*"],
757 |         )
758 |         logger.info("CORS actualizado", origins=cors_origins)
759 |
760 |     # Verificar disponibilidad de módulos
761 |     if enable_pipeline and not PIPELINE_AVAILABLE:
762 |         logger.warning("Pipeline solicitado pero no disponible")
763 |
764 |     if enable_monitoring and not MONITORING_AVAILABLE:
765 |         logger.warning("Monitoring solicitado pero no disponible")
766 |
767 |     logger.info(
768 |         "API lista para uso",
769 |         version=__version__,
770 |         pipeline_enabled=enable_pipeline and PIPELINE_AVAILABLE,
771 |         monitoring_enabled=enable_monitoring and MONITORING_AVAILABLE
772 |     )
773 |
774 |     return app
775 |
776 | except Exception as e:
777 |     logger.error(
778 |         f"Error configurando API: {e}",
779 |         exc_info=True,
780 |         enable_pipeline=enable_pipeline,
781 |         enable_monitoring=enable_monitoring
782 |     )
783 |     raise
784 |
785 |
786 | # Exportar funciones y clases principales
787 | __all__ = [
788 |     'app',
789 |     'create_api_app',
790 |     'get_available_modules',
791 |     'MemoryStoreRequest',
792 |     'MemoryRetrieveRequest',
793 |     'RedundancyProcessRequest',
794 |     'PipelineProcessRequest',
795 |     'ChatRequest',
796 |     'PIPELINE_AVAILABLE',
797 |     'CONFIG_MANAGER_AVAILABLE',
798 |     'MONITORING_AVAILABLE',
799 |     '__version__'
800 | ]

```

```

801 |
802 |
803 | if __name__ == "__main__":
804 |     uvicorn.run(app, host="0.0.0.0", port=8000)
805 |
806 |

```